

# MYSQL TERMINAL — COMPLETE CLEAN NOTES

**Purpose:** A command-focused reference for working in MySQL Shell and SQL mode. This is based on the supplied terminal record and the existing MySQL notes. Database/table creation and project data-loading are intentionally not included here.

## 1. Start MySQL Shell

```
mysqlsh
```

Open MySQL Shell from Windows Command Prompt/Terminal.

## 2. Switch to SQL Mode

```
\sql
```

Changes MySQL Shell from JavaScript mode to SQL mode.

## 3. Connect to MySQL Server

```
\connect root@localhost
```

Creates a session with the local MySQL server. If required, MySQL asks for the password.

## 4. Select the Existing Database

```
\use banking_db
```

Selects the already-created banking database for the current session.

## 5. Check Current Database

```
SELECT DATABASE();
```

Shows which database/schema is currently selected.

## 6. View Available Databases

```
SHOW DATABASES;
```

## 7. View Tables

```
SHOW TABLES;
```

Shows all tables inside the currently selected database.

## 8. Check Table Structure

```
DESC customers;
```

```
DESCRIBE customers;
```

DESC and DESCRIBE show columns, data types, NULL information, keys, defaults and extra information.

## 9. Read Data with SELECT

```
SELECT * FROM customers;
```

```
SELECT customer_id, first_name, last_name, email  
FROM customers;
```

## 10. WHERE — Filter Rows

```
SELECT *
FROM accounts
WHERE balance > 100000;
```

## 11. Comparison Operators

| Operator | Meaning               |
|----------|-----------------------|
| =        | Equal                 |
| >        | Greater than          |
| <        | Less than             |
| >=       | Greater than or equal |
| <=       | Less than or equal    |
| <> / !=  | Not equal             |

## 12. AND / OR

```
SELECT *
FROM accounts
WHERE balance > 100000
AND account_type = 'SAVINGS';
```

```
SELECT *
FROM accounts
WHERE account_type = 'SAVINGS'
OR account_type = 'CURRENT';
```

## 13. IN

```
SELECT *
FROM accounts
WHERE account_type IN ('SAVINGS', 'CURRENT');
```

IN is a short form for checking multiple possible values.

## 14. BETWEEN

```
SELECT *
FROM accounts
WHERE balance BETWEEN 50000 AND 200000;
```

## 15. LIKE and Wildcards

```
SELECT *
FROM customers
WHERE first_name LIKE 'A%';
```

```
SELECT *
FROM customers
WHERE first_name LIKE '%a';
```

% = zero or more characters    \_ = exactly one character.

## 16. NULL

```
SELECT *
FROM customers
WHERE email IS NULL;
```

```
SELECT *
FROM customers
WHERE email IS NOT NULL;
```

Do not use = NULL. Use IS NULL or IS NOT NULL.

## 17. ORDER BY

```
SELECT *  
FROM accounts  
ORDER BY balance ASC;
```

```
SELECT *  
FROM accounts  
ORDER BY balance DESC;
```

ASC = ascending; DESC = descending.

## 18. LIMIT

```
SELECT *  
FROM accounts  
ORDER BY balance DESC  
LIMIT 3;
```

## 19. Aggregate Functions

| Function | Purpose           | Example      |
|----------|-------------------|--------------|
| COUNT()  | Count rows/values | COUNT(*)     |
| SUM()    | Total             | SUM(balance) |
| AVG()    | Average           | AVG(balance) |
| MAX()    | Maximum           | MAX(balance) |
| MIN()    | Minimum           | MIN(balance) |

```
SELECT COUNT(*) AS total_customers FROM customers;
SELECT SUM(balance) AS total_balance FROM accounts;
SELECT AVG(balance) AS average_balance FROM accounts;
SELECT MAX(balance) AS highest_balance FROM accounts;
SELECT MIN(balance) AS lowest_balance FROM accounts;
```

## 20. GROUP BY

```
SELECT account_type,
       COUNT(*) AS total_accounts
FROM accounts
GROUP BY account_type;
```

GROUP BY creates groups based on one or more columns.

## 21. HAVING

```
SELECT account_type,
       COUNT(*) AS total_accounts
FROM accounts
GROUP BY account_type
HAVING COUNT(*) > 2;
```

WHERE filters individual rows; HAVING filters grouped results.

## 22. UPDATE

```
UPDATE customers
SET customer_status = 'INACTIVE'
WHERE customer_id = 5;
```

**Golden rule:** Always check the WHERE condition before UPDATE.

## 23. DELETE

```
DELETE FROM customers
WHERE customer_id = 5;
```

**Golden rule:** Always check the WHERE condition before DELETE.

## 24. Multi-line SQL Query

```
SELECT customer_id,
       first_name,
       last_name,
       email
FROM customers
WHERE customer_status = 'ACTIVE';
```

The query can span multiple lines. The semicolon completes the SQL statement.

## 25. MySQL Prompt and ->

```
MySQL localhost:33060+ ssl banking_db SQL > SELECT *
-> FROM customers
-> WHERE customer_status = 'ACTIVE';
```

The -> prompt means MySQL Shell is waiting for the rest of the current SQL statement.

## 26. Clear the Windows Terminal

```
\system cls
```

Clears only the visible Windows terminal screen. It does not delete the database, tables or data.

## 27. Help

```
\help\n\n\?
```

## 28. Exit MySQL Shell

```
\quit
```

Use this to leave MySQL Shell.

## 29. Complete Daily Workflow

1. Open Command Prompt / Terminal
2. mysqlsh
3. \sql
4. \connect root@localhost
5. \use banking\_db
6. SELECT DATABASE();
7. SHOW TABLES;
8. DESC table\_name;
9. Run required SQL queries
10. \system cls (only when you want to clear the screen)
11. \quit

## 30. SQL Query Order — Must Remember

```
SELECT
FROM
WHERE
GROUP BY
HAVING
ORDER BY
LIMIT;
```

Shortcut: **SELECT → FROM → WHERE → GROUP BY → HAVING → ORDER BY → LIMIT**

## 31. MySQL Shell Commands vs SQL Commands

| MySQL Shell command     | SQL statement            |
|-------------------------|--------------------------|
| \sql                    | SELECT ...;              |
| \connect root@localhost | SHOW TABLES;             |
| \use banking_db         | DESC customers;          |
| \system cls             | SELECT * FROM customers; |
| \help / \?              | UPDATE ...;              |
| \quit                   | DELETE ...;              |

Shell commands use the backslash form (\...), while normal SQL statements are written as SQL and normally end with ;.

## 32. Final Revision Checklist

- ✓ Correct database selected? → `SELECT DATABASE();`
- ✓ Correct table? → `SHOW TABLES;`
- ✓ Correct column names? → `DESC table_name;`
- ✓ Text values inside single quotes?
- ✓ SQL statement ends with `;'?`
- ✓ `UPDATE` has the intended `WHERE` condition?
- ✓ `DELETE` has the intended `WHERE` condition?
- ✓ `NULL` checked with `IS NULL / IS NOT NULL?`
- ✓ Pattern search uses `LIKE?`
- ✓ Multiple values use `IN` where appropriate?
- ✓ Aggregate filtering uses `HAVING?`
- ✓ `GROUP BY` uses the intended grouping column?
- ✓ Sorting uses `ASC/DESC` correctly?
- ✓ `LIMIT` used when only top N rows are needed?

**Important:** These notes are for terminal usage and SQL querying after the database has already been created. They intentionally do not repeat the banking database's `CREATE DATABASE`, `CREATE TABLE`, or bulk `INSERT` script.